

Kosmocrates HYPHAE

Master Specification

v0.2 – Topology-Guided Corpus Assimilation Layer

Author: Sebastian Klemm
30. May 2026

Document status. This document is a normative companion specification to the *Kosmocrates Workbench Master Specification v0.1 – Wonderlamp Distillation Edition*. It replaces the earlier Harpoon v0.1 companion by neutralizing the biological metaphor, renaming the subsystem, and upgrading the crude Wonderlamp topology extraction into a Kosmocrates-native, HDAG-backed, evidence-bound corpus assimilation architecture. The legacy word "Harpoon" is retained only as historical implementation context.

Contents

I	Refounding	1
1	From Harpoon to HYPHAE	2
1.1	Why the earlier name is retired	2
1.2	Meaning of HYPHAE	2
1.3	Metaphor boundary	2
2	Why HYPHAE Requires Kosmocrates	4
2.1	Wonderlamp’s substrate approximation	4
2.2	Responsibility split	4
3	Current Wonderlamp Topology: Findings and Limitations	5
3.1	What the prototype actually computes	5
3.2	Why it is insufficient	5
3.3	Correct interpretation	6
II	Target Architecture	7
4	System Overview	8
4.1	HYPHAE in the Kosmocrates stack	8
4.2	Canonical pipeline	8
5	Core Objects	9
6	Topology Assimilation Model	11
6.1	From CodeObservation to CodeHDAG	11
6.2	Topological signature	11
7	Deficiency-Driven Search	13
7.1	Why host deficiency matters	13
7.2	Deficiency classes	13
7.3	Query generation	13
8	Source Frontier and Acquisition	15
8.1	Keyspaces	15
8.2	Capability lock requirement	15
8.3	Sandbox discipline	15
III	Assimilation and Gates	16
9	Trust, Taint, License, and Injection Controls	17
9.1	Authority model	17
9.2	Prompt-injection structural rule	18

10 Structural Yield	19
10.1 Yield classes	19
10.2 Yield payloads	19
11 Assimilation Decisions	20
11.1 Decision classes	20
11.2 Gate cascade	20
IV Integration with Kosmocrates	21
12 Memory Integration	22
12.1 Kosmocrates objects	22
13 Workbench Integration	23
13.1 ContextPack contribution	23
14 ETV and Retrieval Integration	24
V Implementation Specification	25
15 Module Map	26
15.1 Recommended Rust module structure	26
15.2 Dependency rules	26
16 Public API	27
16.1 REST endpoints	27
16.2 Run request	27
16.3 Run status	27
17 CLI	28
18 Metrics	29
18.1 Core metrics	29
18.2 Success criteria	29
19 Implementation Phases	30
19.1 Phase A: Rename and boundary cleanup	30
19.2 Phase B: Kosmocrates substrate binding	30
19.3 Phase C: CodeHDAG upgrade	30
19.4 Phase D: Assimilation gates	30
19.5 Phase E: Workbench and Foundry integration	30
19.6 Phase F: Advanced retrieval and negative learning	30
20 Acceptance Tests	31
20.1 Minimum tests	31
20.2 Integration tests	31
21 Relationship to the Workbench Master Specification	32
22 What v0.2 Changes from v0.1	33
23 Definition of Done	34
Closing Thesis	35

HYPHAE is not a scraper. It is a topology-guided corpus assimilation layer for Kosmocrates.

HYPHAE stands for:

Hypergraph-based Yielding Pattern Harvest and Assimilation Engine.

It is the Kosmocrates-native evolution of the legacy Harpoon mechanism. It binds a host repository, computes its structural deficiency vector, constructs a governed source frontier, retrieves external code corpora under capability locks, converts acquired material into CodeObservations, lowers those observations into Kosmocrates HDAG/topological signatures, extracts reusable structural patterns, and only then emits candidate knowledge for Workbench use.

The biological imagery is non-normative. The normative architecture is:

```
Host Repository
-> Host Binding and Genesis Context
-> CodeObservation Graph
-> Kosmocrates CodeHDAG
-> DeficiencyVector
-> SourceFrontier
-> Sandboxed Acquisition
-> Topology Assimilation
-> PatternCandidate / NormCandidate / EvidenceOnly
-> Gate Cascade
-> Kosmocrates Memory / Workbench ContextPack / Agenda
```

Core correction from v0.1

The earlier Harpoon v0.1 described the current Wonderlamp code as if the topology digestion layer were already close to sufficient. It is not. The repository implementation computes a simple signature from function name/arity, struct names, path-derived layers, import edges, language counts, and an approximate edge-density connectivity score. That is useful as a prototype, but insufficient for the final system.

The final HYPHAE architecture must use Kosmocrates as the actual epistemic substrate: HDAG, Infinity Ledger, QTIC/Q-class evidence, ETV retrieval, causal topology, authority/taint classification, capability locks, replay packs, and Foundry validation.

Central doctrine

External code may contribute evidence and structure. It may not directly become trusted code, trusted memory, trusted governance, or authorized action.

Design consequence

HYPHAE is useful only because Kosmocrates now exists. In Wonderlamp, Harpoon attempted to simulate a substrate locally through norm extraction and norm crystallization. In Kosmocrates, that responsibility moves into the persistent epistemic substrate itself. HYPHAE becomes a frontier and assimilation layer, not a standalone memory engine.

Part I

Refounding

1 From Harpoon to HYPHAE

1.1 Why the earlier name is retired

The name *Harpoon* is operationally vivid, but misleading as a normative architecture name. A harpoon suggests a single pointed strike. The actual mechanism is not a single strike. It is a recursive, target-bound, topology-guided corpus traversal and assimilation process.

The subsystem needs a name that captures:

- distributed traversal through source keyspaces;
- host-bound extraction rather than generic scraping;
- structural harvest instead of direct code copying;
- assimilation into a governed memory substrate;
- compatibility with Kosmocrates terminology.

Requirement Name-001: Normative subsystem name. The normative name of the subsystem is **HYPHAE**. The phrase “legacy Harpoon” **MAY** be used in implementation notes, migration logs, and backwards-compatible CLI aliases, but **MUST NOT** be used as the primary name in new architecture documents.

1.2 Meaning of HYPHAE

HYPHAE = Hypergraph-based Yielding Pattern Harvest and Assimilation Engine
--

This name keeps a compact trace of the user’s original hyphal intuition without placing a biological metaphor on the system surface. HYPHAE is professional enough for a specification, short enough for code, and distinctive enough to name a subsystem.

1.3 Metaphor boundary

The user described the mechanism through a fungus analogy: a host repository is held as a workpiece, and the system grows through global keyspaces to draw in useful material. This is valid as a mental model, but it must be translated into neutral operational terms.

Intuitive image	Neutral term	Normative meaning
Host organism	Host Repository	Project being analyzed or improved
Hyphae	Source Frontier Edges	Search/acquisition paths through external corpora
Mycelium	Frontier Graph	Replayable graph of queries, sources, clones, observations
Nutrients	Structural Yield	Extracted topology, norms, motifs, tests, interfaces, failures
Digestion	Topology Assimilation	Parser + HDAG lowering + evidence extraction
Growth	Frontier Expansion	Gate-bound generation of new search targets

Assimilation	Candidate Promotion	Evidence-bound movement into Kosmocrates memory
--------------	---------------------	--

Invariant MET-001: Metaphor non-normativity. Biological language **MAY** appear in examples, comments, or explanatory prose. It **MUST NOT** define system behavior. All normative semantics **MUST** be expressed through typed data, gates, transitions, evidence bundles, and Kosmocrates memory operations.

2 Why HYPHAE Requires Kosmocrates

2.1 Wonderlamp's substrate approximation

Wonderlamp attempted to create a local memory and norm-crystallization substrate for software generation. Its norm extraction and norm crystallization mechanisms were meaningful, but they were trying to solve a general epistemic problem inside one application-specific system.

The current state is better understood as follows:

Wonderlamp Norm System: Repository observations -> lightweight topology -> norm candidates -> local norm registry
Kosmocrates Substrate: Observations -> Crystals -> Infinity Ledger -> HDAG -> QTIC conformance -> causal retrieval -> epistemic agenda -> governance-controlled memory

The former is a prototype. The latter is the substrate.

2.2 Responsibility split

Layer	Responsibility
HYPHAE	Find, acquire, parse, lower, and propose external structural material.
Kosmocrates	Persist, govern, retrieve, validate, replay, certify, reject, or promote material.
Workbench	Use gate-passed material for local project work.
Foundry	Prove that generated/modified code compiles, tests, and remains structurally coherent.
Human operator	Authorize goals, accept reports, and approve high-risk transitions.

Invariant SUB-001: HYPHAE is not a memory substrate. HYPHAE **MUST NOT** implement a separate trusted memory system. It **MUST** emit structured candidate evidence into Kosmocrates and let Kosmocrates decide persistence, promotion, retrieval, and replay semantics.

Invariant SUB-002: No direct norm authority. A pattern discovered through HYPHAE **MUST NOT** become a trusted norm by discovery alone. It **MUST** pass Kosmocrates evidence, replay, taint, license, topology, and Foundry-related gates before any future run may treat it as trusted structure.

3 Current Wonderlamp Topology: Findings and Limitations

3.1 What the prototype actually computes

The repository implementation in `isls-code-topo` computes a `CodeTopology` object from parsed `CodeObservations`. The signature contains:

- node count as functions plus structs/tables;
- edge count as import edges;
- function signatures in the form `name/arity`;
- struct/class/type names;
- path-derived layer labels;
- import graph pairs (`importer_file`, `imported_module`);
- per-language line counts and file counts;
- primary language;
- approximate connectivity as edge density.

Similarity is currently a weighted Jaccard-style combination:

```
0.40 * function_signature_similarity
+ 0.25 * struct_name_similarity
+ 0.20 * layer_similarity
+ 0.15 * size_similarity
```

This is useful for a prototype because it is deterministic, simple, fast, and language-light. However, it is not enough for final HYPHAE.

3.2 Why it is insufficient

The current topology extraction is simplified in several ways:

1. **Name dependence.** It relies heavily on function and type names. Equivalent code with different naming may not match.
2. **Weak graph semantics.** Imports are treated as edges, but call relations, type relations, dataflow, trait/interface implementation, module containment, test-target relations, and API-route relations are not fully represented.
3. **Path-based layer detection.** Layer classification is primarily string/path heuristic. It is useful but not epistemically sufficient.
4. **Approximate connectivity.** Edge density is not a real spectral invariant.
5. **Limited cross-language invariance.** Cross-language recognition is aspirational without a richer common representation.
6. **No full provenance object.** Observations are not yet bound into Kosmocrates-grade `EvidenceBundles` with replay packs and gate traces.
7. **No authority/taint/licensing graph.** External material is not fully modeled as untrusted source material with taint propagation.

3.3 Correct interpretation

The prototype is not wrong. It is an initial probe. It proves that the old system was reaching toward something real: topological extraction from external code corpora. But its extraction must be upgraded to Kosmocrates-grade structure.

Invariant TOPO-001: Prototype extraction is non-authoritative. Legacy CodeTopology values **MUST** be treated as low-resolution discovery signals, not as trusted final structural signatures.

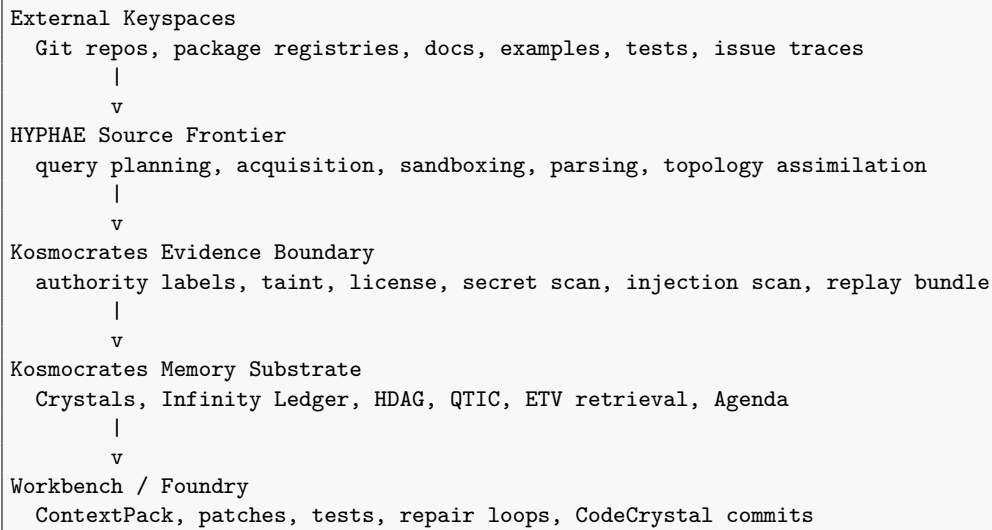
Invariant TOPO-002: Final signatures require Kosmocrates lowering. A source artifact discovered by HYPHAE **MUST** be lowered into a Kosmocrates-compatible CodeHDAG and evidence bundle before it may influence trusted memory or Workbench generation.

Part II

Target Architecture

4 System Overview

4.1 HYPHAE in the Kosmocrates stack



4.2 Canonical pipeline

```
H0 Bind Host
H1 Compute Host Structural State
H2 Compute DeficiencyVector
H3 Generate SourceFrontier Plan
H4 Acquire Sources under Capability Locks
H5 Parse and Normalize Sources
H6 Lower to CodeHDAG
H7 Compute Topological Signatures
H8 Extract Structural Yield
H9 Gate Yield: source, license, taint, injection, secrets, originality
H10 Emit CandidateEvidence
H11 Kosmocrates Assimilation Decision
H12 Workbench Context or Agenda Update
```

Requirement PIPE-001: Host-bound operation. Every HYPHAE run **MUST** bind to a host repository or explicit target project. Generic scraping without a host deficiency vector is out of scope for the default Workbench mode.

Requirement PIPE-002: Kosmocrates mediation. No output from HYPHAE **MAY** directly alter the host repository, Workbench memory, governance rules, or tool authorization state. All such effects **MUST** pass through Kosmocrates.

5 Core Objects

Type HostProject.

```
pub struct HostProject {  
  pub id: Digest,  
  pub root_path: PathBuf,  
  pub vcs_head: Option<String>,  
  pub workspace_digest: Digest,  
  pub primary_languages: BTreeMap<String, usize>,  
  pub genesis_crystal_id: Digest,  
  pub policy_profile_id: Digest,  
}
```

Type HostStructuralState.

```
pub struct HostStructuralState {  
  pub host_id: Digest,  
  pub observations_digest: Digest,  
  pub code_hdag_digest: Digest,  
  pub norm_configuration_digest: Option<Digest>,  
  pub test_coverage_summary: CoverageSummary,  
  pub dependency_summary: DependencySummary,  
  pub known_defects: Vec<DefectRef>,  
  pub unresolved_agenda_items: Vec<AgendaItemId>,  
}
```

Type DeficiencyVector.

```
pub struct DeficiencyVector {  
  pub host_id: Digest,  
  pub goal_id: Digest,  
  pub missing_capabilities: Vec<CapabilityClass>,  
  pub weak_layers: Vec<LayerDeficiency>,  
  pub unstable_edges: Vec<HDAGEdgeRef>,  
  pub missing_tests: Vec<TestGap>,  
  pub unknowns: Vec<UnknownSlot>,  
  pub priority: BTreeMap<DeficiencyKind, f64>,  
}
```

Type SourceFrontierPlan.

```
pub struct SourceFrontierPlan {  
  pub id: Digest,  
  pub host_id: Digest,  
  pub deficiency_vector_id: Digest,  
  pub queries: Vec<SourceQuery>,  
  pub source_policy: SourcePolicy,  
  pub acquisition_budget: AcquisitionBudget,  
  pub expected_yields: Vec<ExpectedYield>,  
}
```

Type SourceQuery.

```
pub struct SourceQuery {  
  pub id: Digest,  
  pub query_string: String,  
}
```

```

    pub target_keyspace: Keyspace,
    pub derived_from: Vec<DerivationRef>,
    pub expected_yield: ExpectedYield,
    pub risk_class: SourceRiskClass,
}

```

Type AcquiredSource.

```

pub struct AcquiredSource {
    pub id: Digest,
    pub uri: String,
    pub commit_or_version: Option<String>,
    pub local_sandbox_path: PathBuf,
    pub source_digest: Digest,
    pub license_report: LicenseReport,
    pub taint_label: TaintLabel,
    pub acquisition_trace_id: Digest,
}

```

Type CodeHDAG.

```

pub struct CodeHDAG {
    pub id: Digest,
    pub source_id: Digest,
    pub nodes: Vec<CodeNode>,
    pub edges: Vec<CodeEdge>,
    pub graph_digest: Digest,
    pub spectral_signature: SpectralSignature,
    pub role_signature: RoleSignature,
    pub provenance: EvidenceBundleId,
}

```

Type StructuralYield.

```

pub struct StructuralYield {
    pub id: Digest,
    pub source_id: Digest,
    pub hdag_digest: Digest,
    pub yield_kind: YieldKind,
    pub payload: YieldPayload,
    pub relevance: RelevanceScore,
    pub novelty: NoveltyScore,
    pub evidence: EvidenceBundleId,
    pub taint: TaintLabel,
    pub license_status: LicenseStatus,
}

```

Type CandidateEvidence.

```

pub enum CandidateEvidence {
    PatternCandidate(StructuralYield),
    NormCandidate(StructuralYield),
    AntiPatternEvidence(StructuralYield),
    TestMotifCandidate(StructuralYield),
    InterfaceMotifCandidate(StructuralYield),
    DependencyMotifCandidate(StructuralYield),
    EvidenceOnly(StructuralYield),
}

```

6 Topology Assimilation Model

6.1 From CodeObservation to CodeHDAG

Legacy Wonderlamp used `CodeObservation` values as a useful input. HYPHAE keeps this, but changes what happens next. Observations are not the final representation. They are an intermediate parse product that must be lowered into a `CodeHDAG`.

Algorithm Lowering pipeline.

```
for each acquired source S:
    parse all supported files into CodeObservation[]
    normalize paths, language tags, spans, symbols
    create CodeNodes:
        File, Module, Function, Type, Trait, Interface, Route, Test,
        Table, Query, Service, Config, Dependency, BuildTarget
    create CodeEdges:
        contains, calls, imports, exports, implements, instantiates,
        returns, consumes, provides, tests, migrates, configures,
        routes_to, queries, serializes, validates, depends_on
    compute canonical graph digest
    compute spectral/topological metrics
    attach provenance and source taint
    emit CodeHDAG
```

Requirement HDAG-001: Typed edges. Code topology **MUST** use typed edges. Untyped lists of imports or Jaccard string features **MAY** be retained as auxiliary discovery features, but **MUST NOT** be treated as the canonical structural representation.

Requirement HDAG-002: Span-aware observations. Where parser support exists, nodes **SHOULD** include file path, byte span, line span, language, visibility, declaration kind, signature, and stable symbol id.

Requirement HDAG-003: Test relation extraction. The topology assimilation layer **SHOULD** extract explicit relations between tests and targets when names, imports, fixtures, or calls allow it.

6.2 Topological signature

A final signature is not a single similarity scalar. It is a multi-axis object:

```
pub struct SpectralSignature {
    pub node_count: usize,
    pub edge_count: usize,
    pub connected_components: usize,
    pub spectral_gap: Option<Q16>,
    pub fiedler_profile: Option<Vec<Q16>>,
    pub role_entropy: Q16,
    pub layer_balance: BTreeMap<LayerKind, Q16>,
    pub dependency_density: Q16,
    pub test_coupling: Q16,
    pub api_surface_complexity: Q16,
```

```
    pub instability_score: Q16,  
}
```

Requirement SIG-001: Multi-axis signatures. HYPHAE **MUST** compute or request from Kosmocrates a multi-axis CodeHDAG signature. A single Jaccard score **MUST NOT** be used as the final basis for trusted assimilation.

Requirement SIG-002: Deterministic arithmetic. Where numerical metrics influence gates, the implementation **SHOULD** use deterministic fixed-point or otherwise replay-stable arithmetic compatible with Kosmocrates replay rules.

7 Deficiency-Driven Search

7.1 Why host deficiency matters

HYPHAE is not a global crawler. It starts from a host and a goal. The goal determines the kind of external material that is useful.

Examples:

- Missing tests -> search for similar projects' test structures.
- Unstable API layer -> search for route/handler/service separation motifs.
- Weak database migration pattern -> search for migration/query/repository motifs.
- Missing auth -> search for authentication norm evidence, not raw auth code.
- Compile repair loop failure -> search for examples matching the error class and dependency set.

7.2 Deficiency classes

```
pub enum DeficiencyKind {
    MissingLayer,
    WeakLayer,
    MissingInterface,
    BrokenInterface,
    MissingTest,
    LowCoverage,
    CompileFailurePattern,
    RuntimeFailurePattern,
    SecurityGap,
    DependencyGap,
    DocumentationGap,
    UnknownArchitecturalClass,
    NormConfigurationGap,
}
```

7.3 Query generation

```
queries = derive_queries(
    host_topology,
    deficiency_vector,
    workbench_goal,
    prior_kosmocrates_retrieval,
    failed_foundry_logs,
    known_norm_gaps,
    allowed_keyspaces,
    source_policy
)
```

Requirement QUERY-001: Query provenance. Every source query **MUST** record why it was generated. Acceptable derivations include host symbol, missing capability, failed test class, dependency gap, ETV retrieval hit, norm gap, or operator-specified target.

Requirement QUERY-002: No hallucinated authority. A query string generated by an LLM **MUST** be treated as a proposal. It may not launch acquisition until a SourceFrontierPlan passes policy gates.

8 Source Frontier and Acquisition

8.1 Keyspaces

```
pub enum Keyspace {  
    GitRepositorySearch,  
    PackageRegistry,  
    DocumentationCorpus,  
    IssueTracker,  
    TestCorpus,  
    LocalCorpus,  
    ApprovedMirror,  
}
```

The default implementation may start with Git repository search, but the architecture is broader.

8.2 Capability lock requirement

Acquisition is a privileged operation. It performs network access, downloads untrusted material, executes parsing, and may consume resources.

Gate SRC-001: Source acquisition capability. No external source acquisition **MAY** begin unless a SourceAcquisitionCapability is unlocked under the active Genesis Crystal and policy profile.

```
pub struct SourceAcquisitionCapability {  
    pub allowed_keyspaces: Vec<Keyspace>,  
    pub max_sources: usize,  
    pub max_bytes: u64,  
    pub max_depth: usize,  
    pub allowed_domains: Vec<String>,  
    pub forbidden_domains: Vec<String>,  
    pub requires_license_detection: bool,  
    pub requires_secret_scan: bool,  
    pub expires_at: Option<Timestamp>,  
}
```

8.3 Sandbox discipline

Requirement ACQ-001: No direct host writes. Acquired sources **MUST** be stored in an isolated acquisition sandbox. They **MUST NOT** be copied into the host repository except through a separate, explicit, human-approved import workflow.

Requirement ACQ-002: Shallow and bounded by default. Repository acquisition **SHOULD** use shallow clones or archive downloads where possible, bounded by source count, byte count, timeout, and depth policy.

Requirement ACQ-003: Reproducible source identity. Each acquired source **MUST** bind URI, commit/version if available, timestamp, content digest, license report, and acquisition trace.

Part III

Assimilation and Gates

9 Trust, Taint, License, and Injection Controls

9.1 Authority model

External source material is not trusted. It begins as `ExternalContent`. It may become `EvidenceOnly`, `PatternCandidate`, or `NormCandidate`; it may not become trusted memory or executable output without gates.

```
pub enum SourceAuthority {
    HostTrustedCode,
    HostUncommittedDiff,
    ApprovedInternalCorpus,
    ExternalRepository,
    ExternalDocumentation,
    ExternalIssueThread,
    UnknownExternal,
}

pub enum TaintLabel {
    CleanHost,
    InternalApproved,
    ExternalUntrusted,
    LicenseRestricted,
    SecretAdjacent,
    InjectionCandidate,
    SuspiciousGeneratedCode,
    Quarantined,
}
```

Gate TRUST-001: Taint propagation. Taint labels **MUST** propagate from `AcquiredSource` to `CodeHDAG` to `StructuralYield` to `CandidateEvidence`. Promotion may reduce taint only through explicit gate reports, never by omission.

Gate LICENSE-001: Non-copying default. External code **MUST** be treated as non-copyable by default. HYPHAE's normal output is structural evidence, not source-code import. Raw copying requires a distinct license-aware, human-approved import path.

Gate SECRET-001: Secret scan. Candidate material containing suspected secrets, credentials, tokens, private keys, environment values, or secret-adjacent configuration **MUST** be quarantined and **MUST NOT** enter Workbench ContextPacks except as redacted evidence.

Gate INJECT-001: Injection scan. Textual content from external sources **MUST** be scanned and labeled for instruction-like content. Instruction-like text may be summarized as evidence, but **MUST NOT** be placed into prompts as higher-authority instructions.

9.2 Prompt-injection structural rule

External source material may inform HYPHAE. It may not instruct Kosmocrates.

Forbidden transitions:

```
ExternalSource -> SystemInstruction
ExternalSource -> ToolCapability
ExternalSource -> GovernanceRule
ExternalSource -> TrustedMemory
ExternalSource -> DirectPatch
ExternalSource -> SecretAccess
```

10 Structural Yield

10.1 Yield classes

```
pub enum YieldKind {
    ArchitectureMotif,
    LayerSeparationPattern,
    InterfacePattern,
    TestPattern,
    ErrorHandlingPattern,
    DependencyPattern,
    BuildConfigurationPattern,
    SecurityPattern,
    APISurfacePattern,
    DataModelPattern,
    AntiPattern,
    UnknownPattern,
}
```

10.2 Yield payloads

```
pub enum YieldPayload {
    CodeHDAGFragment(CodeHDAGFragment),
    NormCandidate(NormCandidateDraft),
    MetaNormEvidence(MetaNormEvidenceDraft),
    TestMotif(TestMotifDraft),
    InterfaceMotif(InterfaceMotifDraft),
    AntiPattern(AntiPatternDraft),
    ContextHint(ContextHint),
}
```

Requirement YIELD-001: No opaque text yield. Structural Yield **MUST** be typed. An opaque text blob **MUST NOT** be promoted as a PatternCandidate. If textual documentation is useful, it **MUST** be attached as attributed evidence.

Requirement YIELD-002: Host alignment. Each StructuralYield **MUST** explain how it aligns with the host deficiency vector. Low-alignment yields may be stored as EvidenceOnly but **MUST NOT** influence Workbench generation by default.

Requirement YIELD-003: Novelty and duplication. Before promotion, a yield **MUST** be compared against Kosmocrates memory for duplication, near-duplication, conflict, and novelty.

11 Assimilation Decisions

11.1 Decision classes

```
pub enum AssimilationDecision {  
    Reject { reason: RejectReason },  
    Quarantine { reason: QuarantineReason },  
    EvidenceOnly { crystal_id: Digest },  
    CandidatePattern { candidate_id: Digest },  
    CandidateNorm { candidate_id: Digest },  
    ContextPackEligible { context_item_id: Digest },  
    AgendaItem { agenda_id: Digest },  
}
```

11.2 Gate cascade

```
G0 Genesis and policy profile valid  
G1 Source acquisition capability valid  
G2 Source identity and digest verified  
G3 License report acceptable for structural use  
G4 Secret scan clean or redacted  
G5 Injection taint handled  
G6 Parser and CodeHDAG lowering successful  
G7 Topological signature computed  
G8 Host relevance above threshold or EvidenceOnly  
G9 Novelty/conflict check passed  
G10 Kosmocrates memory promotion policy passed  
G11 Optional Foundry validation for executable patterns  
G12 Crystal / candidate / agenda emission
```

Gate ASSIM-001: Fail closed. If any gate cannot evaluate due to missing evidence, the default decision is Quarantine or EvidenceOnly, not CandidatePattern or CandidateNorm.

Gate ASSIM-002: Executable influence requires Foundry. A discovered pattern that may influence generated executable code **SHOULD** pass a Foundry-backed validation path before becoming high-trust memory. Until then it remains candidate evidence.

Part IV

Integration with Kosmocrates

12 Memory Integration

12.1 Kosmocrates objects

HYPHAE emits into existing Kosmocrates concepts rather than creating a separate memory system.

HYPHAE output	Kosmocrates target
Acquisition trace	EvidenceBundle / Infinity Ledger event
CodeHDAG	HDAG node group / CodeCrystal candidate
StructuralYield	EvidenceOnly Crystal or PatternCandidate
NormCandidate	Normic Pattern Memory candidate, not trusted norm
AntiPattern	Agenda Guard item / Risk memory
ContextHint	Workbench ContextPack candidate
DeficiencyVector	Epistemic Agenda and Workbench planning input
Search frontier	Replayable SearchLedger

Requirement MEM-001: One persistent substrate. All durable HYPHAE outcomes **MUST** be persisted through Kosmocrates. No second trusted database of learned patterns may exist.

Requirement MEM-002: Evidence-first persistence. The first durable representation of external material **SHOULD** be evidence, not norm. Norms are promotions from evidence.

13 Workbench Integration

13.1 ContextPack contribution

HYPHAE contributes to Workbench only through bounded, attributed, typed ContextPack items.

```
pub struct HyphaeContextItem {
  pub id: Digest,
  pub source_yield_id: Digest,
  pub host_deficiency_id: Digest,
  pub kind: ContextItemKind,
  pub summary: String,
  pub structural_payload_ref: Digest,
  pub taint: TaintLabel,
  pub permitted_use: PermittedUse,
}

pub enum PermittedUse {
  ReadOnlyEvidence,
  PromptContextSummary,
  StructuralScaffoldHint,
  TestDesignHint,
  HumanReviewRequired,
}
```

Requirement CTX-001: No raw external code in default prompts. Default Workbench prompts **MUST NOT** include raw external code obtained by HYPHAE. They **MAY** include attributed summaries, structural signatures, pseudocode-like neutral abstractions, or schema-level motifs if gates allow.

Requirement CTX-002: Use limits. Every HYPHAE-derived ContextPack item **MUST** declare permitted use. The Workbench Agent **MUST** enforce this value.

14 ETV and Retrieval Integration

HYPHAE should use Kosmocrates retrieval to plan external acquisition, and its discoveries should improve future retrieval.

Before acquisition:

ETV retrieves prior patterns, failed searches, known norm gaps, source policy, host history.

After assimilation:

HYPHAE emits evidence-backed candidates and negative results into memory.

Requirement ETV-001: Retrieval before frontier expansion. Before launching external acquisition, HYPHAE **SHOULD** query Kosmocrates for existing relevant patterns. If internal memory suffices, external acquisition **SHOULD** be skipped or reduced.

Requirement ETV-002: Negative result persistence. Failed searches, irrelevant sources, license-rejected sources, and low-quality patterns **SHOULD** be persisted as negative evidence to reduce repeated waste.

Part V

Implementation Specification

15 Module Map

15.1 Recommended Rust module structure

```
crates/  
  kosmo-hyphae/  
    src/  
      lib.rs  
      host.rs  
      deficiency.rs  
      frontier.rs  
      acquisition.rs  
      parsing.rs  
      hdag.rs  
      yield_extract.rs  
      gates.rs  
      assimilation.rs  
      context_pack.rs  
      api.rs  
      metrics.rs  
      policy.rs  
      tests/  
  kosmo-hyphae-cli/  
  kosmo-hyphae-gateway/ (optional if gateway remains separate)
```

If Kosmocrates prefers fewer crates, **hyphae** may be a module inside the Workbench crate. The critical point is not crate count but boundary clarity.

15.2 Dependency rules

- HYPHAE may depend on Kosmocrates core types, evidence, HDAG, governance, and Workbench context types.
- HYPHAE may use parser libraries and tree-sitter frontends behind deterministic wrappers.
- HYPHAE must not depend on an LLM provider.
- HYPHAE must not own trusted memory persistence.
- HYPHAE must not bypass Workbench/Foundry gates for generated code.

16 Public API

16.1 REST endpoints

```
POST /api/hyphae/runs
GET  /api/hyphae/runs/{id}
GET  /api/hyphae/runs/{id}/events
GET  /api/hyphae/runs/{id}/frontier
GET  /api/hyphae/runs/{id}/yields
POST /api/hyphae/runs/{id}/stop
POST /api/hyphae/yields/{id}/promote
POST /api/hyphae/yields/{id}/quarantine
GET  /api/hyphae/policy
```

16.2 Run request

```
pub struct HyphaeRunRequest {
    pub host_path: PathBuf,
    pub goal: Option<String>,
    pub deficiency_hint: Option<Vec<DeficiencyKind>>,
    pub max_depth: Option<usize>,
    pub max_sources: Option<usize>,
    pub keyspaces: Option<Vec<Keyspace>>,
    pub policy_profile: Option<String>,
    pub dry_run: bool,
}
```

16.3 Run status

```
pub struct HyphaeRunStatus {
    pub id: Digest,
    pub status: RunStatus,
    pub host_id: Digest,
    pub depth_current: usize,
    pub sources_considered: usize,
    pub sources_acquired: usize,
    pub sources_rejected: usize,
    pub yields_extracted: usize,
    pub candidates_emitted: usize,
    pub quarantined: usize,
    pub coverage_delta: Option<f64>,
    pub stop_reason: Option<String>,
}
```

17 CLI

```
kosmocrates hyphae run --host . --goal "add robust auth tests"
kosmocrates hyphae run --host . --deficiency missing-tests --max-sources 20
kosmocrates hyphae status <run-id>
kosmocrates hyphae frontier <run-id>
kosmocrates hyphae yields <run-id>
kosmocrates hyphae promote <yield-id> --as evidence-only
kosmocrates hyphae promote <yield-id> --as pattern-candidate
```

Legacy aliases may exist:

```
kosmocrates harpoon run ... # deprecated alias for kosmocrates hyphae run
```

Requirement CLI-001: Alias warning. If legacy harpoon aliases are implemented, they **SHOULD** print a deprecation notice identifying HYPHAE as the canonical command.

18 Metrics

18.1 Core metrics

```
pub struct HyphaeMetrics {  
    pub sources_acquired: usize,  
    pub sources_rejected: usize,  
    pub avg_acquisition_ms: f64,  
    pub parse_success_rate: f64,  
    pub code_hdag_success_rate: f64,  
    pub yield_acceptance_rate: f64,  
    pub quarantine_rate: f64,  
    pub license_rejection_rate: f64,  
    pub secret_rejection_rate: f64,  
    pub injection_candidate_rate: f64,  
    pub workbench_reuse_rate: f64,  
    pub foundry_validation_rate: f64,  
    pub autonomy_delta: f64,  
}
```

18.2 Success criteria

A HYPHAE run is not successful merely because it scraped many repositories. Success is measured by useful, safe, evidence-backed assimilation.

Metric	Meaning
High parse success	Acquisition sources are compatible and processable.
High quarantine precision	Unsafe material is caught without rejecting everything.
High reuse rate	Extracted yields actually help Workbench tasks.
Low repeated search waste	Negative evidence is being used.
Positive autonomy delta	Future similar tasks need less external/LLM effort.
Foundry validation rate	Executable motifs become working code only after real validation.

19 Implementation Phases

19.1 Phase A: Rename and boundary cleanup

1. Introduce **hyphae** naming in APIs, CLI, and docs.
2. Keep **harpoon** as deprecated alias only.
3. Remove normative mycelial/nutrient language from code-facing structs.
4. Define HostProject, DeficiencyVector, SourceFrontierPlan, StructuralYield.

19.2 Phase B: Kosmocrates substrate binding

1. Bind HYPHAE runs to Genesis Crystal and policy profile.
2. Emit EvidenceBundles for acquisition and parsing.
3. Persist results only through Kosmocrates memory APIs.
4. Add taint propagation and license/secret/injection gates.

19.3 Phase C: CodeHDAG upgrade

1. Replace simple CodeTopology as final representation.
2. Add typed CodeNodes and CodeEdges.
3. Add spectral/topological signature contract.
4. Add parse-back and replay checks.

19.4 Phase D: Assimilation gates

1. Implement gate cascade G0-G12.
2. Add Quarantine/EvidenceOnly/PatternCandidate/NormCandidate decisions.
3. Add Workbench ContextPack permitted-use enforcement.

19.5 Phase E: Workbench and Foundry integration

1. Let TaskSpec request HYPHAE assistance when memory is insufficient.
2. Feed selected ContextItems into Workbench.
3. Run Foundry validation for executable candidates.
4. Emit CodeCrystals only after compile/test/replay closure.

19.6 Phase F: Advanced retrieval and negative learning

1. Use ETV retrieval before external search.
2. Persist negative search and rejection evidence.
3. Track autonomy delta over repeated tasks.

20 Acceptance Tests

20.1 Minimum tests

```
AT-HYP-001 host binding produces stable HostProject digest
AT-HYP-002 deficiency vector generated from missing test layer
AT-HYP-003 SourceFrontierPlan records query provenance
AT-HYP-004 acquisition requires capability lock
AT-HYP-005 acquired source stored outside host repository
AT-HYP-006 license-restricted source cannot become PatternCandidate
AT-HYP-007 secret-containing source is quarantined
AT-HYP-008 injection-like external text cannot enter prompt as instruction
AT-HYP-009 CodeObservation lowers into typed CodeHDAG
AT-HYP-010 simple legacy CodeTopology is treated as auxiliary only
AT-HYP-011 StructuralYield cannot be opaque text
AT-HYP-012 EvidenceOnly persistence works without memory promotion
AT-HYP-013 NormCandidate requires replayable evidence
AT-HYP-014 Workbench ContextItem enforces permitted use
AT-HYP-015 raw external code excluded from default prompt context
AT-HYP-016 Foundry validation required before executable high-trust pattern
AT-HYP-017 negative search result is persisted and reused
AT-HYP-018 harpoon alias prints deprecation warning
AT-HYP-019 dry-run mode performs no acquisition
AT-HYP-020 run report includes all gate decisions
```

20.2 Integration tests

```
IT-HYP-001 local host repo with missing CRUD tests:
  HYPHAE discovers external test motifs, emits TestMotifCandidate,
  Workbench uses permitted structural hint, Foundry tests pass.

IT-HYP-002 external repo with incompatible license:
  source is parsed, evidence stored, but pattern promotion is blocked.

IT-HYP-003 external README contains prompt injection text:
  text is labeled InjectionCandidate and cannot affect system instruction.

IT-HYP-004 repeated task:
  second run uses Kosmocrates memory and launches fewer external searches.
```

21 Relationship to the Workbench Master Specification

HYPHAE v0.2 is a companion, not a replacement. It plugs into the Workbench Master Spec at these points:

Workbench chapter	HYPHAE contribution
Kosmocrates Core Binding ADAMANT / Genesis Authority/Taint/Capability Workspace Agent	HYPHAE emits evidence into the same substrate. All HYPHAE runs bind to Genesis and policy. Source acquisition and assimilation are capability-locked. Agent can request HYPHAE assistance when memory is insufficient.
Forge / Foundry	HYPHAE candidates inform structure; Foundry validates executable effects.
Normic Pattern Memory	HYPHAE supplies external evidence for NormCandidates and MetaNorms.
Code Topology / ArtifactIR	HYPHAE provides CodeHDAG fragments and structural signatures.
Evaluation / Telemetry	HYPHAE contributes source, yield, quarantine, reuse, and autonomy metrics.

22 What v0.2 Changes from v0.1

Topic	v0.1	v0.2
Name	Harpoon	HYPHAE; Harpoon legacy alias only
Surface language	mycelial/nutrient heavy	neutral technical vocabulary
Current topology	treated as roughly adequate	explicitly marked low-resolution prototype
Memory model	Harpoon emits nutrients	HYPHAE emits candidate evidence into Kosmocrates
Substrate role	partly standalone assimilation	no trusted memory outside Kosmocrates
Code representation	CodeTopology/NutrientPacket	CodeHDAG/StructuralYield/CandidateEvidence
Security	evidence-gated	stronger authority/taint/license/secret/injection gates
Prompt use	summaries and nutrients	permitted-use ContextItems; no raw external code by default
Implementation plan	broad architecture	phased rename, substrate binding, HDAG upgrade, gate cascade

23 Definition of Done

HYPHAE v0.2 is implemented when:

1. **hyphae** is the canonical module/API/CLI name, with **harpoon** only as deprecated alias.
2. Every run binds to HostProject, Genesis Crystal, and policy profile.
3. External acquisition requires a capability lock and executes only inside sandbox boundaries.
4. Acquired sources are digested into CodeObservations and lowered into CodeHDAG.
5. Legacy CodeTopology is no longer the authoritative signature.
6. StructuralYield is typed, evidenced, taint-labeled, and license-labeled.
7. License, secret, injection, taint, and provenance gates fail closed.
8. Raw external code is excluded from default Workbench prompts.
9. Candidate promotion occurs only through Kosmocrates memory APIs.
10. Foundry validation is required before high-trust executable influence.
11. Negative search and rejection evidence is persisted.
12. Metrics report source count, yield count, quarantine rate, reuse rate, and autonomy delta.
13. All AT-HYP and IT-HYP tests pass.

Closing Thesis

HYPHAE is the evolved form of the legacy Harpoon idea. It no longer pretends that a simple scrape-and-norm loop can provide its own epistemic foundation. That role belongs to Kosmocrates. HYPHAE's task is narrower and stronger: bind a host, discover relevant external structure, assimilate it into CodeHDAG evidence, and let Kosmocrates decide what may become memory, context, pattern, norm, agenda, or nothing.

HYPHAE traverses external keyspaces. Kosmocrates decides what can become knowledge.